

Relier textes, profils et synthèse contextualisée avec NaileR

Dernière case du parcours

Objectif de cette dernière case

Cette dernière case montre comment **NaileR** peut combiner deux sources d'information :

- les **verbatim**s des individus ;
- les **variables structurées** qui décrivent les groupes.

L'objectif n'est pas seulement de produire une interprétation à partir de textes. Il est de montrer comment on peut construire deux artefacts complémentaires, puis les réunir dans une synthèse contextualisée.

Le cheminement est le suivant :

```
textes par groupe  
→ artefact textuel  
→ profil structuré des groupes  
→ synthèse contextualisée
```

Charger NaileR

On commence par charger le package.

```
library(NaileR)
```

Dans cet exemple, les données sont simulées. Elles portent sur des attitudes vis-à-vis du covoiturage et de la mobilité partagée.

Créer un petit jeu de données synthétique

On crée trois groupes fictifs :

- des conducteurs pragmatiques ;
- des individus écologiquement flexibles ;
- des individus attachés à leur autonomie.

```

set.seed(123)

n <- 90

group <- sample(
  x = c("Pragmatic drivers", "Eco-flexible", "Autonomy first"),
  size = n,
  replace = TRUE,
  prob = c(0.35, 0.30, 0.35)
)

```

Ces groupes ne sont pas observés à partir d'une vraie enquête. Ils servent ici à illustrer la logique de la dernière case.

Générer des variables structurées

On crée ensuite quelques variables descriptives : lieu de résidence, habitude de transport et type d'emploi du temps.

Ces variables sont volontairement liées au groupe pour rendre l'exemple lisible.

```

residence <- sapply(group, function(g) {
  if (g == "Pragmatic drivers") {
    sample(c("Urban", "Peri-urban"), 1, prob = c(0.6, 0.4))
  } else if (g == "Eco-flexible") {
    sample(c("Urban", "Peri-urban", "Rural"), 1, prob = c(0.5, 0.3, 0.2))
  } else {
    sample(c("Rural", "Peri-urban"), 1, prob = c(0.7, 0.3))
  }
})

transport_habit <- sapply(group, function(g) {
  if (g == "Pragmatic drivers") {
    sample(c("Car", "Mixed"), 1, prob = c(0.75, 0.25))
  } else if (g == "Eco-flexible") {
    sample(c("Mixed", "Public transport", "Bike"), 1, prob = c(0.45, 0.30, 0.25))
  } else {
    sample(c("Car", "Mixed"), 1, prob = c(0.85, 0.15))
  }
})

schedule_type <- sapply(group, function(g) {
  if (g == "Pragmatic drivers") {
    sample(c("Variable", "Fixed"), 1, prob = c(0.7, 0.3))
  } else if (g == "Eco-flexible") {
    sample(c("Fixed", "Variable"), 1, prob = c(0.65, 0.35))
  } else {
    sample(c("Variable", "Fixed"), 1, prob = c(0.75, 0.25))
  }
})

```

```
}  
})
```

Générer une variable textuelle

Chaque individu reçoit maintenant un court commentaire. Le texte est cohérent avec son groupe.

Cela permet d'avoir un matériau textuel simple pour tester les fonctions de NaileR.

```
make_text <- function(g, res, habit, sched) {  
  if (g == "Pragmatic drivers") {  
    base <- sample(c(  
      "Carpooling can be useful, but in daily life time constraints make it difficult.",  
      "I see environmental benefits, but I need something practical and easy to organize.",  
      "I could share some journeys, but only when the schedule is simple.",  
      "The main issue is efficiency: I do not want to lose time in everyday travel."  
    ), 1)  
  
    add <- sample(c(  
      "My schedule changes often, so I cannot always adapt to others.",  
      "If the trip is planned in advance, I am more open to sharing.",  
      "Public transport is available, but it is not always the fastest option.",  
      "I mainly think in terms of convenience and feasibility."  
    ), 1)  
  
  } else if (g == "Eco-flexible") {  
    base <- sample(c(  
      "Reducing emissions matters to me, so I am generally favorable to shared mobility.",  
      "I try to limit solo car use whenever I can.",  
      "Carpooling makes sense environmentally and socially.",  
      "I am quite open to alternatives to driving alone."  
    ), 1)  
  
    add <- sample(c(  
      "If the organization is reasonable, I am willing to adapt.",  
      "I already combine several transport options depending on the situation.",  
      "For regular trips, collective solutions seem realistic.",  
      "The environmental argument is important in my decisions."  
    ), 1)  
  
  } else {  
    base <- sample(c(  
      "I need autonomy in my journeys and I do not want to depend on other people's schedules.",  
      "My car gives me freedom, especially when trips are irregular.",  
      "In theory carpooling is interesting, but in practice I need independence.",  
      "I prefer keeping control over departure times and route changes."  
    ), 1)  
  
    add <- sample(c(  

```

```

    "Living outside dense urban areas makes flexibility essential.",
    "Unexpected events make shared travel complicated.",
    "My daily organization depends on being able to leave when I want.",
    "I am not against the idea, but autonomy comes first."
  ), 1)
}

context <- paste(
  "Residence:", res, "|",
  "Habit:", habit, "|",
  "Schedule:", sched, "."
)

paste(base, add, context)
}

text_answer <- mapapply(
  FUN = make_text,
  g = group,
  res = residence,
  habit = transport_habit,
  sched = schedule_type,
  SIMPLIFY = TRUE
)

```

Construire le tableau final

Le tableau contient une variable de groupe, une variable textuelle et plusieurs variables descriptives.

```

dta_text <- data.frame(
  Group = factor(group),
  Text = text_answer,
  Residence = factor(residence),
  TransportHabit = factor(transport_habit),
  ScheduleType = factor(schedule_type),
  stringsAsFactors = FALSE
)

str(dta_text)
table(dta_text$Group)

```

La variable **Group** est la variable qui définit les groupes à comparer. La variable **Text** contient les commentaires libres.

Première étape : `nail_textual()`

La fonction `nail_textual()` prépare un prompt à partir des textes regroupés par modalité de **Group**.

Ici, on met `generate = FALSE` pour inspecter le prompt sans appeler de modèle de langage.

```
res_textual_prompt <- nail_textual(  
  dataset = dta_text,  
  num.var = 1, # Group  
  num.text = 2, # Text  
  generate = FALSE  
)  
  
names(res_textual_prompt)  
attr(res_textual_prompt, "textual_data_summary")  
  
cat(res_textual_prompt[[1]])
```

À ce stade, on ne demande pas encore au modèle de produire une interprétation. On vérifie d'abord ce qu'on va lui transmettre.

Génération optionnelle avec `nail_textual()`

Si un modèle local est disponible, on peut générer une réponse.

```
# res_textual <- nail_textual(  
#   dataset = dta_text,  
#   num.var = 1,  
#   num.text = 2,  
#   model = "llama3",  
#   generate = TRUE  
# )
```

Deuxième étape : `nail_textual_prep()`

La fonction `nail_textual_prep()` prépare un artefact textuel par groupe.

Elle peut repérer les thèmes principaux, des verbatims représentatifs et des expressions notables.

```
res_textual_prep <- nail_textual_prep(  
  dataset = dta_text,  
  num.var = 1,  
  num.text = 2,  
  model = "llama3",  
  generate = TRUE  
)  
  
names(res_textual_prep)
```

On peut inspecter le résultat pour un groupe.

```
res_textual_prep[[1]]$parsed
res_textual_prep[[1]]$selected_verbatims
res_textual_prep[[1]]$notable_expressions
```

Cet objet est un premier artefact. Il résume ce que les textes disent de chaque groupe.

Troisième étape : `nail_group_profile_prep()`

Les textes ne sont pas la seule information disponible.

La fonction `nail_group_profile_prep()` prépare un profil des groupes à partir des variables structurées du tableau.

```
res_group_profile <- nail_group_profile_prep(
  dataset = dta_text,
  num.var = 1,
  model = "llama3",
  generate = TRUE
)

names(res_group_profile)
res_group_profile[[1]]$parsed
```

On obtient ici un second artefact. Il décrit les groupes à partir des variables comme `Residence`, `TransportHabit` ou `ScheduleType`.

Quatrième étape : `nail_textual_contextualized()`

La dernière fonction combine les deux artefacts :

- l'artefact textuel ;
- l'artefact structuré.

L'objectif est de produire une synthèse plus contextualisée.

```
res_contextualized_prompt <- nail_textual_contextualized(
  group_profile_prep = res_group_profile,
  textual_prep = res_textual_prep,
  interpretation_mode = "comparative",
  generate = FALSE
)

cat(res_contextualized_prompt)
```

Avec `generate = FALSE`, on inspecte le prompt final. C'est utile pour vérifier que la synthèse repose bien sur les deux sources d'information.

Génération finale optionnelle

Si le modèle local est disponible, on peut demander la synthèse finale.

```
# res_contextualized <- nail_textual_contextualized(  
#   group_profile_prep = res_group_profile,  
#   textual_prep = res_textual_prep,  
#   interpretation_mode = "comparative",  
#   model = "llama3",  
#   generate = TRUE  
# )  
#  
# res_contextualized$response
```

À retenir

Cette dernière case montre la logique complète du workflow textuel avec NaileR.

On part de commentaires libres, mais on ne les interprète pas isolément.

On construit d'abord un artefact textuel. Puis on construit un artefact structuré à partir des variables du tableau. Enfin, on combine les deux dans une synthèse contextualisée.

L'intérêt est de garder une trace claire du raisonnement :

```
verbatim  
→ profil textuel  
→ profil structuré  
→ synthèse contextualisée
```

Cette logique est importante pour garder le contrôle sur l'interprétation produite par le modèle de langage.

Le modèle n'interprète pas seul. Il travaille à partir d'artefacts préparés, visibles et inspectables.